

Aula 05 — Árvores de Regressão e *Ensembles*

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: AME §4.8–4.9; ISLP cap. 8

O que você vai aprender

Ao final desta aula você deve ser capaz de:

- descrever como uma **árvore de regressão** particiona o espaço de covariáveis e prediz por médias locais;
- explicar a construção em duas etapas: *crescimento* por divisões binárias recursivas e *poda* por custo-complexidade;
- justificar, via redução de variância, por que **agregar** estimadores ajuda;
- descrever **bagging** e explicar como as **florestas aleatórias** *descorrelacionam* as árvores, e o papel de m .

Árvores são o método mais *interpretável* que veremos — e, sozinhas, das piores em predição. A história desta aula é como transformá-las, por agregação, em um dos métodos *mais poderosos* que existem para dados tabulares: as florestas aleatórias. A pergunta que organiza tudo:

como combinar muitos modelos ruins para obter um ótimo?

A resposta tem nome e sobrenome — Proposição 2.1 — e duas condições. Cada método desta aula é uma tentativa de satisfazer melhor a segunda delas.

1 Árvores de regressão

Uma árvore parte o espaço de covariáveis em regiões disjuntas $R_1, \dots, R_J$ por *particionamento recursivo* e prediz, em cada região, a média das respostas de treino ali contidas:

$$g(\mathbf{x}) = \frac{1}{|\{i : \mathbf{X}_i \in R_k\}|} \sum_{i: \mathbf{X}_i \in R_k} Y_i, \quad \text{para } \mathbf{x} \in R_k. \quad (1)$$

Para prever, percorre-se a árvore do topo às folhas seguindo as condições (“ $x_j < t$?”) em cada nó.

Ideia central

Vantagens estruturais das árvores: (i) altamente **interpretáveis**; (ii) lidam *nativamente* com covariáveis discretas (cortes do tipo $x_j \in \{\text{verde}, \text{vermelho}\}$ — sem precisar criar *dummies*); (iii) fazem **seleção de variáveis** automática; (iv) capturam **interações** sem que precisemos especificá-las.

Atenção: e a desvantagem estrutural

Os cortes são sempre *perpendiculares aos eixos*. Uma fronteira diagonal simples, como $x_1 + x_2 > 10$, obriga a árvore a aproximá-la por uma escadinha de muitos cortes — e cada degrau custa dados. Se você suspeita de relações lineares nas covariáveis, um modelo

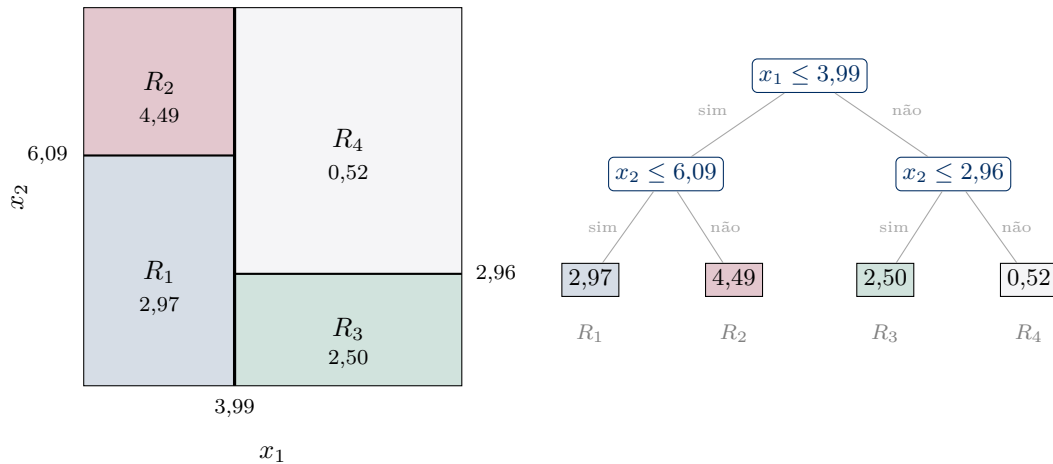


Figura 1: Os dois lados da mesma coisa. À esquerda, a partição de $[0, 10]^2$ em quatro retângulos; à direita, a árvore que a produz. Cada nó interno é um corte perpendicular a um eixo, cada folha é uma região, e o número na folha é a média (1) das respostas de treino ali. Os valores são de um ajuste real ($n = 300$, profundidade 2, $R^2 = 0,941$ no treino). Repare que o corte de x_2 é *diferente* nos dois ramos (6,09 à esquerda, 2,96 à direita): é assim que a árvore captura uma interação sem que ninguém a tenha especificado.

linear (Aula 02) faz melhor com muito menos.

1.1 Etapa 1 — crescimento (divisões binárias recursivas)

Idealmente escolheríamos a partição que minimiza o EQM $P(T) = \sum_R \sum_{i: \mathbf{X}_i \in R} (Y_i - \hat{y}_R)^2 / n$. Mas buscar a árvore ótima é computacionalmente inviável (NP-difícil). Usa-se então uma heurística *gananciosa*: a cada passo, escolhe-se a covariável x_j e o corte t que mais reduzem a soma de quadrados ao dividir uma região em $R_1 = \{x_j < t\}$ e $R_2 = \{x_j \geq t\}$:

$$\min_{j, t} \sum_{i: \mathbf{X}_i \in R_1} (Y_i - \hat{y}_{R_1})^2 + \sum_{i: \mathbf{X}_i \in R_2} (Y_i - \hat{y}_{R_2})^2. \quad (2)$$

Repete-se recursivamente em cada região-filha, até um critério de parada (ex.: menos de 5 observações por folha). Isso gera uma árvore grande T_0 , que ajusta bem o treino mas tende a **superajustar**.

Observação 1.1. “Ganancioso” quer dizer que a escolha de (2) olha *um passo* à frente e nunca volta atrás: o primeiro corte é o que mais reduz o RSS agora, ainda que outro corte pior agora permitisse cortes muito melhores depois. Não é um defeito de implementação — é o preço de fugir de um problema NP-difícil. Guarde isso: parte do ganho dos *ensembles* vem justamente de compensar essa miopia.

Observação 1.2 (o tamanho da miopia, medido). Vale saber quanto isso custa, porque é mais do que parece. Tome $y = \sin(x_1 x_2 x_3)$ em $[-1, 1]^3$: oito regiões resolvem o problema *exatamente*, e caberiam numa árvore de profundidade 3. A §3 da **Aula prática 05** mede o que a árvore gananciosa faz com ele:

- o melhor corte da raiz reduz 0,194% do RSS dali, e nos dois filhos que ele produz os melhores cortes reduzem 0,447% e 0,942% — a cegueira **não** é só do primeiro passo, porque os oito octantes continuam se cancelando dois a dois depois de qualquer corte. Para ter escala, a partição verdadeira em oito octantes deixa RSS zero;

- o RSS de teste fica **acima** do de chutar a média até a profundidade 5, e pior a cada nível — a árvore gasta folhas para ficar mais errada que o preditor constante; cai para 83% dele na 6 e só chega a 13% com a árvore cheia, de 59 folhas.

Ela precisa gastar cinco níveis de cortes inúteis antes de as regiões ficarem pequenas o bastante para o sinal aparecer. A mesma seção do notebook mede o caso de duas dimensões, $y = \text{sin}(x_1 x_2)$, em que basta *um* corte às cegas: cada dimensão a mais custa um nível inteiro de cegueira.

E há um corolário prático que vale mais que o exemplo. Se você tentar economizar cortes com um critério de parada precoce — `min_impurity_decrease=0,005`, digamos — a árvore fica com **uma folha**: o critério nunca vê meio por cento de redução em lugar nenhum, e desiste antes de começar. É por isso que se cresce a árvore inteira e *depois* se poda, que é o assunto da próxima subseção.

1.2 Etapa 2 — poda por custo-complexidade

Para reduzir a variância, *podamos* T_0 . A poda por custo-complexidade introduz um parâmetro $\alpha \geq 0$ e busca a subárvore $T \subseteq T_0$ que minimiza

$$\sum_{k=1}^{|T|} \sum_{i: \mathbf{X}_i \in R_k} (Y_i - \hat{y}_{R_k})^2 + \alpha |T|, \quad (3)$$

onde $|T|$ é o número de folhas. O termo $\alpha |T|$ penaliza árvores grandes — é exatamente a mesma estrutura da regularização da Aula 02, com $|T|$ no papel de $\|\beta\|_1$: ajuste + $\lambda \times$ complexidade. α é o **tuning parameter**, escolhido por **validação cruzada** (Aula 03): $\alpha = 0$ devolve T_0 ; $\alpha \rightarrow \infty$ colapsa para a raiz (média global).

Atenção

Crescer e podar usam objetivos diferentes *de propósito*: cresce-se de forma gananciosa (olhando só um passo à frente) e corrige-se globalmente na poda. Não confunda profundidade máxima (parâmetro de crescimento) com α (parâmetro de poda) — ambos controlam complexidade, mas em etapas distintas.

2 Por que agregar? Redução de variância

Árvores isoladas têm *alta variância*: pequenas mudanças nos dados mudam bastante a árvore — basta que o corte da raiz mude um pouco para que toda a estrutura abaixo dele seja outra. A ideia dos *ensembles* é **combinar** muitas funções para reduzir essa variância.

Proposição 2.1 (Média de preditores reduz o risco). *Sejam $g_1, \dots, g_B$ estimadores não-viesados ($\mathbb{E}[g_b(\mathbf{x})] = r(\mathbf{x})$), de mesma variância $v(\mathbf{x}) = \text{Var}(g_b(\mathbf{x}))$ e não-correlacionados. Então a média $\bar{g} = \frac{1}{B} \sum_b g_b$ tem risco*

$$\mathbb{E}[(Y - \bar{g}(\mathbf{x}))^2 | \mathbf{x}] = \text{Var}(Y | \mathbf{x}) + \frac{v(\mathbf{x})}{B} \leq \mathbb{E}[(Y - g_b(\mathbf{x}))^2 | \mathbf{x}].$$

Demonstração. Pela decomposição viés-variância (Aula 01), o risco de qualquer preditor g é $\text{Var}(Y | \mathbf{x}) + \text{viés}^2 + \text{Var}(g(\mathbf{x}))$. Como cada g_b é não-viesado, $\bar{g}$ também é (viés = 0). Para a variância, sendo os g_b não-correlacionados e de variância $v(\mathbf{x})$, $\text{Var}(\bar{g}(\mathbf{x})) = \frac{1}{B^2} \sum_b \text{Var}(g_b(\mathbf{x})) = v(\mathbf{x})/B$. Substituindo obtém-se a igualdade; como $v(\mathbf{x})/B \leq v(\mathbf{x})$, o risco não aumenta. $\square$

Ideia central

Duas exigências da Prop. 2.1 guiam tudo o que segue: precisamos de preditores (i) **não-viesados** — por isso usamos árvores *não podadas* (profundas, baixo viés) — e (ii) **pouco correlacionados** — o ponto fraco, que as florestas aleatórias atacam diretamente.

Observação 2.2 (o que acontece quando a hipótese falha). A hipótese de não-correlação é forte demais para ser verdade, e vale saber o preço. Se as g_b têm correlação ρ duas a duas, a conta vira

$$\text{Var}(\bar{g}(\mathbf{x})) = \rho v(\mathbf{x}) + \frac{1-\rho}{B} v(\mathbf{x}) \xrightarrow{B \rightarrow \infty} \rho v(\mathbf{x}).$$

Ou seja: por mais árvores que você acrescente, a variância **não desce abaixo de $\rho v(\mathbf{x})$** . O termo que B mata é só o segundo. É por isso que reduzir ρ vale mais do que aumentar B — e é literalmente essa a ideia das florestas aleatórias.

3 Bagging

Bagging (*bootstrap aggregating*) gera diversidade reamostrando os dados. Para $b = 1, \dots, B$: sorteie uma amostra **bootstrap** (com reposição, tamanho n) e ajuste nela uma árvore *profunda* (não podada) g_b . A predição é a média:

$$g(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}). \quad (4)$$

Note a divisão de trabalho: as árvores são profundas (para ter viés baixo, condição (i)) e a média cuida da variância. Sozinha, cada árvore superaajusta descaradamente — e é para isso que ela está ali.

4 Florestas aleatórias

O problema do *bagging*: as árvores ficam **correlacionadas**. Se há uma covariável muito forte, quase todas as árvores a usam no topo e ficam parecidas — e, pela observação acima, a variância trava em $\rho v(\mathbf{x})$ por mais que B cresça.

Ideia central

Florestas aleatórias (Breiman) *descorrelacionam* as árvores: em *cada nó*, sorteia-se um subconjunto de apenas $m < p$ covariáveis e a divisão só pode usar uma delas. Nos nós em que a variável dominante não é sorteada — uma fração $1 - m/p$ deles — a árvore é obrigada a olhar para outra coisa, e descobre estrutura que o *bagging* nunca veria. Menos ρ , menos variância.

Detalhes:

- m é o *tuning parameter*; regra empírica $m \approx p/3$ em regressão (e $m \approx \sqrt{p}$ em classificação). $m = p$ recupera o *bagging*.
- O risco é **robusto a B** : ele estabiliza e *não há superaajuste* ao aumentar B — não vale a pena fazer CV em B , basta usar B “grande o bastante”.
- A taxa de convergência das florestas depende essencialmente do número de *variáveis relevantes* — elas se adaptam à *esparsidade*.

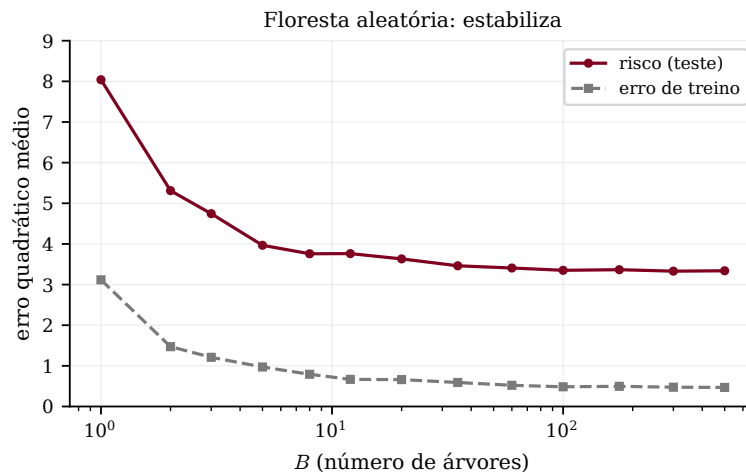


Figura 2: Por que B não é um *tuning parameter* na floresta, medido num problema com $n = 150$, $p = 8$ e ruído alto. Com $B = 1$ — uma árvore só — o risco é 8,04; ele desaba, estabiliza em torno de 3,34 e *fica lá*: de $B = 100$ para $B = 500$ varia $-0,4\%$. O erro de treino também estaciona, em 0,47, em vez de descer rumo a zero — cada árvore nova é ajustada a uma reamostra independente, e a média de mais árvores independentes não persegue ruído. Você pode exagerar em B à vontade; só perde tempo de máquina.

Resumo

- **Árvore:** partição recursiva (1), com os dois lados da Figura 1; cresce gananciosamente (2) e poda por custo-complexidade (3) (α por CV). Interpretável, mas alta variância, e só corta perpendicular aos eixos.
- Agregar reduz variância se os preditores são não-viesados e pouco correlacionados (Prop. 2.1); com correlação ρ , a variância trava em $\rho v(\mathbf{x})$.
- **Bagging** (4): árvores profundas em amostras bootstrap, agregadas pela média.
- **Florestas:** *descorrelacionam* sorteando $m \approx p/3$ variáveis por nó; adaptam-se à esparsidade; e são *robustas a B* — exagerar só custa tempo de máquina (Fig. 2).

Leitura recomendada. [AME] §4.8 (árvores, com o algoritmo de crescimento e poda) e §4.9 (*bagging* e florestas aleatórias). [ISLP] §8.1, que tem a versão deles da nossa Figura 1, e §8.2.1–8.2.2 sobre *bagging* e florestas.